

Guia v4 — useradd (variações) e ssh-copy-id

Introdução

Este documento explica: (1) como usar `useradd` com as opções mais úteis e seus efeitos; (2) como permitir que um usuário recém-criado acesse a VM via SSH usando `ssh-copy-id`.

Observação: `adduser` existe e é mais amigável no Ubuntu, mas aqui focamos em `useradd` por oferecer opções explícitas e previsíveis em scripts de produção.

Parte A — Referência prática: useradd

O comando `useradd` cria contas no sistema. Muitas opções controlam home, shell, grupos, UID/GID e políticas de expiração. Abaixo alguns exemplos.

Alguns parâmetros

- `-m` cria o diretório home e copia arquivos de `/etc/skel`. Ex.: `useradd -m alice`.
- `-M` não cria diretório home mesmo se o sistema padrão criaria.
- `-s /bin/shell` define o shell de login (ex.: `/bin/bash`, `/usr/sbin/nologin`). Ex.: `useradd -s /bin/bash alice`.
- `-g GRUPO` grupo primário - nome ou GID existente. Ex.: `useradd -g deploy -m alice`. O grupo deve existir.
- `-r` system account: cria conta de sistema (UID baixo, sem senha por padrão). Ex.: `useradd -r -s /usr/sbin/nologin -M mysvc`.
- `-e YYYY-MM-DD` define data de expiração da conta. Ex.: `useradd -e 2026-12-31 -m tempuser`.
- `-f DAYS` dias após expiração de senha para desativar a conta.

Exemplos práticos (fluxos comuns)

1) Criar usuário normal com home e bash:

```
sudo useradd -m -s /bin/bash -c "Alice Dev" alice
sudo passwd alice # define senha interativa
```

2) Criar usuário com grupos suplementares e UID fixo (útil em infra com NFS):

```
sudo groupadd -f deploy
sudo useradd -m -s /bin/bash -u 1501 -g deploy -G sudo,adm alice
```

3) Criar conta de sistema sem home (serviço):

```
sudo useradd -r -s /usr/sbin/nologin -M mysvc
```

4) Deletar um usuário e sua pasta home:

```
sudo userdel -r alice
```

Boas práticas em produção

- Prefira scripts que criem o grupo primeiro (se necessário) e definam UIDs conhecidos quando houver armazenamento compartilhado.
- Use `-r` para contas de sistema e defina shells nologin quando apropriado.
- Documente UIDs reservados e políticas de expiração (`-e` , `-f`).

Parte B — Habilitar acesso SSH usando apenas ssh-copy-id

Fluxo mínimo e seguro, adequado para produção quando combinado com um mecanismo de senha temporária ou comunicação segura do administrador:

Pré-requisitos

- O usuário existe no servidor (criado com `useradd` e com home).
- O servidor aceita autenticação por senha temporariamente (ou você tem console para instalar a chave).
- O cliente do usuário tem `ssh` e `ssh-copy-id` disponíveis.

Passo a passo — fluxo recomendado (produção)

1. **(servidor)** Administrador cria a conta e define uma senha temporária segura:

```
sudo useradd -m -s /bin/bash -c "Alice Dev" alice
sudo passwd alice # administrador define senha temporária
```

2. **(servidor)** Administrador configura temporariamente autenticação por senha no servidor (servidores na nuvem vêm por padrão com login e senha desativados)

```
sudo nano /etc/ssh/sshd_config

# edite no arquivo

PasswordAuthentication yes
PermitEmptyPasswords no

# Dependendo da distribuição precisa alterar
# o arquivo de configuração em /etc/ssh/sshd_config.d/
# altere o arquivo .conf que estiver ali
# (no meu caso era o 60-cloudimg-settings.conf)
```

```
sudo nano /etc/ssh/sshd_config/60-cloudimg-settings.conf

PasswordAuthentication yes
```

```
# reinicie o serviço Ubuntu/Debian
sudo systemctl restart ssh

# reinicie o serviço RHEL/CentOS/Amazon Linux
sudo systemctl restart sshd
```

3. **(cliente)** Usuário gera par de chaves no seu computador local:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C "alice@sptech.school"

# ou

ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -C "alice@sptech.school"
```

4. **(cliente)** Usuário copia a chave pública para o servidor usando `ssh-copy-id` :

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@dns_servidor

# Será pedido a senha temporária que o administrador configurou
```

5. **(cliente/servidor)** Verificar login via chave e, depois, (opcional/seguro) o administrador bloqueia a senha para forçar uso exclusivo de chave:

```
# No cliente:
ssh -i ~/.ssh/id_ed25519 alice@dns_servidor

# No servidor (admin) - bloquear senha da conta para prevenir login por senha
sudo passwd -l alice
```

Verificações importantes no servidor

- Permissões:

```
chmod 700 /home/alice/.ssh && chmod 600 /home/alice/.ssh/authorized_keys
chown -R alice:alice /home/alice/.ssh
```

- `sshd_config`: `PubkeyAuthentication yes` e `AuthorizedKeysFile .ssh/authorized_keys` .
- Se `PasswordAuthentication no` estiver em vigor, o usuário não conseguirá usar `ssh-copy-id` — nesse caso use console ou um administrador para instalar a chave.

Verificações rápidas

```
# (servidor) Verificar usuário
getent passwd alice

# (servidor) Verificar as pastas home e .ssh
sudo ls -ld /home/alice /home/alice/.ssh

# (servidor) Verificar permissões de ~/.ssh e ~/.ssh/authorized_keys
sudo -u alice stat -c "%a %U %G %n" /home/alice/.ssh /home/alice/.ssh/authorized_keys

# (servidor) Verificar sshd config relevante
sudo grep -E "PubkeyAuthentication|AuthorizedKeysFile|PasswordAuthentication|AllowGroups" /etc/ssh/sshd_config

# (cliente) Teste de login [-vvv para máximo verbose]
ssh -i ~/.ssh/id_ed25519 -vvv alice@server.example.com
```

Problemas comuns

- **ssh-copy-id falha:** provavelmente porque `PasswordAuthentication` está desabilitado ou a senha temporária não foi comunicada.
- **Permissões incorretas:** .ssh com permissão >700 ou authorized_keys com permissão >600 causa falha de autenticação.

Case de produção

1. Crie um usuário com diretório home para cada integrante do grupo
2. Crie um usuário de serviço que vai rodar o NodeJS sem diretório e sem acesso a bash
3. Configure em cada máquina do grupo uma chave privada para acesso SSH ao servidor
4. Cheque a configuração, permissão e acesso de todos os usuários ao servidor

